

TRABAJO PRÁCTICO N° 1

Minería de Datos II

Proyecto Integrador

Entrega 1: Diseño y fundación de datos

DATOS DE LA ENTREGA:

- Alumna: Cintia Soledad Juarez
 - Docente: Diego Mosquera
 - Fecha de Entrega: 07 de Octubre de 2026
 - Cuatrimestre / Año: 2° Cuatrimestre - 2026
 - Repositorio del Proyecto: [Enlace a Colab](#)
[Enlace a Genially](#)
[Enlace a README](#)
-

1· Interpretación del problema, usuarios, preguntas principales y objetivos medibles.

1.1. Contexto y Problema de Negocio

La empresa es un proveedor de servicios Cloud que genera diariamente una gran cantidad de información relacionada con sus operaciones, facturación y transacciones. Estos datos llegan inicialmente a la zona de aterrizaje (Landing) en su formato original, por lo que pueden presentar datos faltantes, valores incorrectos o atípicos y cambios en la estructura de la información.

Ante esta situación, surge la necesidad de contar con una plataforma que permita centralizar, organizar y procesar estos datos de forma más ordenada y eficiente. Para ello, se propone la implementación de un Data Lake que pueda trabajar tanto con procesos Batch como con datos que llegan en tiempo real mediante Streaming.

El objetivo es disponer de información confiable y accesible que pueda ser utilizada para diferentes necesidades de la empresa, como el control y análisis de costos (FinOps), el seguimiento de problemas y consultas de los usuarios (Soporte Técnico) y el análisis de productos y servicios (Gestión de Producto).

1.2. Usuarios Finales y Casos de Uso

- **FinOps (Finanzas y Operaciones Cloud):** Analiza la facturación consolidada, conversión de monedas a USD, márgenes de ganancia, créditos aplicados e identificación de desviaciones en costos operativos por cliente y servicio.
- **Soporte Técnico:** Monitorea el volumen de incidentes, nivel de severidad, tasa de cumplimiento de SLAs (*Service Level Agreements*) y satisfacción del cliente (CSAT).
- **Gestión de Producto y Sostenibilidad:** Evalúa patrones de consumo de infraestructura (Compute, Storage, Database, GenAI), la demanda de peticiones, consumo de tokens en IA Generativa y métricas de huella de carbono (carbon_kg).

1.3. Preguntas Principales de Negocio

1. ¿Cuál es el consumo acumulado y el costo total en USD por organización, servicio y región en un período determinado?
2. ¿Qué porcentaje de tickets de soporte incumplen el SLA según la severidad y el plan del cliente?
3. ¿Cuál es el impacto ambiental (huella de carbono en carbon_kg) generado por los servicios de IA Generativa y compute por cliente?
4. ¿Qué clientes presentan un puntaje de NPS crítico o atípico que requiera intervención comercial?

1.4. Objetivos Medibles

- **Latencia de Ingesta:** Procesar eventos de uso con una latencia inferior a 5 minutos mediante *Structured Streaming*.

- **Calidad de Datos:** Como objetivo se plantea identificar y aislar los registros corruptos o fuera de rango en la zona Quarantine sin interrumpir el flujo del pipeline.
- **Estandarización Financiera:** Convertir las facturas emitidas en monedas locales (ARS, EUR) a USD utilizando la tasa de cambio histórica correspondiente.

2. Justificación de la necesidad de Big Data.

Se identificaron características que justifican el uso de una arquitectura Big Data. Estas se pueden relacionar con las cinco dimensiones principales.

- **Volumen:** Procesamiento continuo de millones de eventos de consumo e infraestructura recolectados en periodos de 60+ días para múltiples *tenants* o clientes.
- **Velocidad:** Necesidad de ingestar e inspeccionar flujos de uso cerca de tiempo real (*near real-time*) mediante *Structured Streaming* para la detección oportuna de anomalías de costo y fallas de servicio.
- **Variedad:** Convivencia de archivos estructurados (.csv para maestros y facturación) con semiestructurados (.jsonl para métricas streaming). Además, el esquema evoluciona en el tiempo incorporando nuevos atributos sin previo aviso.
- **Veracidad:** Datos en crudo con ruido: registros fuera de escala (p. ej., puntajes NPS con valores -3 o 12), costos negativos, tipos ambiguos (value ingresando como tipo texto) y campos nulos en eventos críticos.
- **Valor:** Los datos obtenidos de los distintos servicios permiten generar indicadores que ayuden a detectar sobrecostos, analizar el comportamiento de los clientes y medir el impacto ambiental.

3. Inventario y perfil inicial de las fuentes: grano, frecuencia, tipos, calidad, trazabilidad y riesgos.

3.1. Tabla de Inventario de Fuentes (Landing)

Fuente	Formato	Tipo Ingesta	Grano del Registro	Frecuencia	Descripción Principal
customers_orgs.csv	CSV	Batch	1 fila por organización	Estática / Maestra	Entidades/inquilinos de la nube (plan,

Fuente	Formato	Tipo Ingesta	Grano del Registro	Frecuencia	Descripción Principal
					industria, región, NPS).
users.csv	CSV	Batch	1 fila por usuario	Maestra	Usuarios vinculados a organizaciones con roles y fechas.
resources.csv	CSV	Batch	1 fila por recurso	Maestra	Catálogo de recursos cloud desplegados (Compute, Storage, GenAI).
support_tickets.csv	CSV	Batch	1 fila por ticket	Diaria / Batch	Registro de incidentes de soporte, SLA, severidad y CSAT.
marketing_touches.csv	CSV	Batch	1 fila por interacción	Diaria / Batch	Historial de puntos de contacto de marketing y conversiones.
nps_surveys.csv	CSV	Batch	1 fila por encuesta	Periódica	Encuestas de satisfacción del cliente en el tiempo.

Fuente	Formato	Tipo Ingesta	Grano del Registro	Frecuencia	Descripción Principal
billing_monthly.csv	CSV	Batch	1 fila por factura/mes	Mensual	Facturas mensuales expresadas en distintas monedas (USD, EUR, ARS).
usage_events_stream/*.jsonl	JSONL	Streaming	1 fila por evento	Tiempo Real	Stream de consumo de métricas, peticiones, tokens y carbono.

3.2. Perfil de Calidad, Trazabilidad y Hallazgos Críticos

Al comienzo del análisis de las fuentes se encontraron algunos problemas que deben ser considerados antes de comenzar el procesamiento.
Los principales hallazgos fueron los siguientes:

- **Evolución Dinámica de Esquema:** En usage_events_stream, los registros con schema_version = 1 omiten campos de sostenibilidad. A partir del 2025-07-18 (schema_version = 2), se introducen las columnas genai_tokens y carbon_kg.
- **Tipos de Datos Ambiguos:** El atributo value en los eventos JSONL llega como StringType (p. ej., "14.9331"), exigiendo una conversión explícita a tipo numérico (DoubleType).
- **Outliers y Nulos:** Se identifican valores fuera de rango permitido en nps_score (valores como -3.0 o 12.0 cuando el rango estándar es 0 a 10), así como costos incrementales negativos (cost_usd_increment < 0).
- **Trazabilidad y Auditoría:** Todos los archivos procesados en la capa Bronze incorporan metadatos técnicos de auditoría (ingest_ts y source_file).

4. Diagrama de arquitectura de alto nivel con fuentes, ingesta, Data Lake, procesamiento, serving, consumo y capacidades transversales.

4.1. Mapeo del Flujo de Arquitectura

Ya con las características de los datos y de las necesidades planteadas, se diseñó una arquitectura que permite recibir información tanto por procesos Batch como por Streaming. El flujo comienza en las fuentes de datos y continúa por las distintas capas del Data Lake hasta llegar a los usuarios finales.

1. Fuentes de Datos: Sistemas transaccionales CRM, sistemas de soporte, plataformas de marketing, archivos de facturación y concentradores de eventos streaming.
2. Ingesta:
 - Batch: Ingesta periódica de archivos CSV mediante PySpark.
 - Streaming: Micro-lotes continuos de eventos JSONL mediante PySpark Structured Streaming.
3. Data Lake (Arquitectura Medallion):
 - Landing Zone: Archivos crudos inmutables (.csv, .jsonl).
 - Bronze Zone: Persistencia en formato columnar Parquet con esquemas explícitos (StructType) y auditoría (ingest_ts, source_file).
 - Silver Zone: Datos limpios, deduplicados, tipificados y con ruteo de inconsistencias a Quarantine.
 - Gold Zone: Data Marts agregados orientados a casos de uso de negocio.
4. Procesamiento: Motor de procesamiento distribuido Apache Spark (PySpark).
5. Serving Layer: Base de datos NoSQL Apache Cassandra / AstraDB modelada bajo la filosofía Query-First.
6. Consumo: Dashboards de BI (Power BI / Tableau), reportes de FinOps y APIs de consulta operacionales.
7. Capacidades Transversales: Gobernanza de datos, catálogo de metadatos, seguridad por roles y monitoreo de pipelines.

5. Selección justificada del patrón batch, Lambda, Kappa o híbrido.

A partir del análisis realizado, se decidió utilizar una Arquitectura Híbrida / Lambda, ya que las fuentes de origen presentan diferentes necesidades de procesamiento:

- **Capa Batch (Lote):** Diseñada para procesar datos consolidados que cambian con baja frecuencia (maestros de clientes, usuarios, facturación mensual, soporte y marketing). Garantiza consistencia estricta y procesamiento determinista en ventanas de tiempo programadas.
- **Capa Streaming (Tiempo Real):** Diseñada para la ingesta de eventos de consumo e infraestructura (usage_events_stream). Utiliza *Structured Streaming* con *Watermarking* y *Checkpoints* para garantizar procesamiento de baja latencia (*near real-time*), tolerancia a fallos y detección temprana de anomalías de costo.

Se aplica Batch para las fuentes que no necesitan actualizarse continuamente, como la facturación mensual o los datos maestros. En cambio, los eventos de consumo se procesan mediante Streaming porque requieren una respuesta cercana al tiempo real.

6. Mapeo de requisitos a componentes, incluyendo la relación entre las 5V y las decisiones de arquitectura.

6.1. Matriz Requisito vs Componente y Relación con las 5V

Requisito de Negocio	Dimensión 5V	Componente Técnico	Decisión de Arquitectura
Normalización de facturas a USD	Variedad / Veracidad	PySpark Batch	Conversión multi-moneda en capa Silver/Gold (billing_usd).
Monitoreo de uso cerca de tiempo real	Velocidad / Volumen	Structured Streaming	Ingesta continua a Bronze/Silver con Parquet y Checkpoints.
Control de calidad y nulos	Veracidad	PySpark Validation Rules	Ruteo automático de registros corruptos a Quarantine.
Indicadores de Sostenibilidad y GenAI	Variedad / Valor	Schema Evolution Handler	Parsing condicional de esquemas v1/v2 en Bronze.
Consultas de baja latencia para FinOps	Valor	AstraDB / Cassandra	Modelado NoSQL <i>Query-First</i> en la capa Serving.

7. Diseño del Data Lake: zonas, formatos, particiones, naming, retención, metadatos y reglas de promoción.

7.1. Estructura de Carpetas del Data Lake

```
/datalake/
├── landing/      # Datos crudos inmutables (.csv, .jsonl)
├── bronze/      # Formato Parquet, esquema explícito, auditoría (ingest_ts, source_file)
├── silver/      # Formato Parquet, datos limpios, deduplicados y tipificados
│   ├── quarantine/ # Registros inválidos o corruptos aislados para análisis
│   └── gold/      # Data Marts agregados por caso de uso analítico
```

7.2. Convención de Nombres (Naming Convention)

- **Carpetas/Tablas:** [zona]/[dominio]_[entidad] (ej. bronze/customers_orgs, gold/org_daily_usage_by_service).
- **Columnas:** Formato snake_case (ej. org_id, cost_usd_increment).

7.3. Formatos y Estrategia de Particionamiento

- **Formato:** Apache Parquet para todas las zonas procesadas (Bronze, Silver, Gold).
- **Particionamiento:**
 - usage_events: Particionado por year, month, day.
 - billing_monthly: Particionado por year y month.
 - Tablas maestras pequeñas (customers, users): Sin particionamiento para evitar el problema de small files.

7.4. Reglas de Promoción entre Zonas

- **Landing → Bronze:** Validación de estructura contra StructType explícito, adición de metadatos ingest_ts y source_file, conversión a Parquet.
- **Bronze → Silver:** Casteo de tipos, eliminación de duplicados por clave primaria, aislamiento de nulos/outliers hacia Quarantine y normalización de monedas.
- **Silver → Gold:** Agregación de métricas de negocio por grano analítico y construcción de vistas consolidadas.

8. Flujo de datos batch y streaming desde las fuentes hasta las salidas, con herramientas específicas.

1. Ingesta Batch (Fuentes CSV):

Archivos maestros colocados en /landing/ → Leídos por PySpark Batch con esquema StructType → Enriquecidos con ingest_ts y source_file → Guardados en Parquet en /bronze/.

2. Ingesta Streaming (Eventos JSONL):

Archivos .jsonl depositados en /landing/usage_events_stream/ → Leídos con spark.readStream / spark.read con esquema unificado → Procesados por PySpark Structured Streaming → Guardados en /bronze/usage_events con *Checkpoints* activos.

3. Flujo de Salida:

Capa Silver remueve datos corruptos → Capa Gold agrega indicadores → Conector PySpark Spark-Cassandra procesa la carga hacia las tablas de **AstraDB / Cassandra** para consumo de usuario

9. Flujo batch de referencia expresado con lógica MapReduce o equivalente conceptual.

Aunque PySpark abstrae el cómputo en un DAG, el procesamiento responde al modelo de ejecución MapReduce:

1. Fase Map (Transformaciones Locales / Narrow Dependencies):

- **Parsing y Proyección:** Cada executor lee un bloque Parquet/CSV, extrae los atributos definidos y aplica el esquema explícito.
- **Mapeo:** Casteo del atributo value de String a Double, parseo de fechas y generación de metadatos de auditoría (ingest_ts).
- **Filtrado:** Evaluación de reglas básicas de limpieza (cost_usd_increment >= 0).

2. Fase Shuffle / Combine (Wide Dependencies):

- Redistribución distribuida de datos a través de la red agrupando por claves compuestas como (org_id, service, month).

3. Fase Reduce (Agregaciones / Salida):

- Cómputo de agregaciones agregadas (SUM(cost_usd_increment), COUNT(ticket_id)) y persistencia distribuida en el Data Lake.

10. Supuestos, riesgos iniciales, mitigaciones y decisiones todavía abiertas.

10.1. Supuestos Principales

1. Las tasas de cambio expresadas en billing_monthly.csv reflejan el valor promedio representativo del período facturado.
2. Los eventos con schema_version = 1 previos a la introducción de carbon_kg se interpretan con valor cero o nulo controlado en reportes de sostenibilidad sin invalidar el registro.
3. La zona Landing garantiza almacenamiento inmutable de archivos de origen.

10.2. Riesgos Iniciales y Mitigaciones

- **Riesgo 1: Duplicación de eventos en el flujo Streaming.**

- *Mitigación:* Deduplicación basada en event_id y ventana temporal mediante *Watermarking* en PySpark.
- **Riesgo 2: Degradación de rendimiento por archivos pequeños (*Small Files Problem*).**
 - *Mitigación:* Ajuste del número de particiones con `.coalesce()` o re-compactación periódica en la capa Silver/Gold.
- **Riesgo 3: Inyección de datos corruptos o valores atípicos.**
 - *Mitigación:* Aislamiento automático de registros que fallen las reglas de validación hacia la zona de quarantine.

10.3. Decisiones Todavía Abiertas

- Elección de la estrategia final de compactación para los micro-lotes en la zona Bronze Streaming.
- Definición exacta de las claves de partición en Cassandra/AstraDB para optimizar consultas de muy baja latencia en dashboards de FinOps.

11. Estimación preliminar de esfuerzo, roles y recursos requeridos.

11.1. Responsabilidades del Perfil Individual

Al tratarse de un desarrollo individual, asumo la ejecución integral de las siguientes funciones:

- **Data Architecture:** Diseño conceptual del Data Lake Medallion, selección del patrón híbrido (Batch/Streaming) y estructura de particionamiento.
- **Data Engineering:** Desarrollo e implementación de los pipelines en PySpark, definición de esquemas explícitos (StructType), control de checkpoints y persistencia en Parquet.
- **Data Quality & Analytics:** Definición de reglas de validación de datos, control de esquemas v1/v2 y diseño del modelo analítico.

11.2. Roles y Recursos Requeridos

- **Entorno de Cómputo:** Google Colab con Apache Spark (PySpark).
- **Almacenamiento (Data Lake):** Google Drive (carpetas Landing, Bronze, Silver, Gold y Checkpoints).
- **Control de Versiones:** [GitHub](#)

12. Repositorio inicial con README, convenciones y evidencia mínima de lectura, generación o exploración de datos.

[Click Aquí](#)